softstack

Smart Contract Code Review And Security Analysis Report

CUSTOMER	Lambdaplex
SCOPE	Vault, Order Hook & Settlement
DATE	01.06.2026

Made in Germany by softstack GmbH

Table of Contents

1. Disclaimer

The audit makes no statements or warrantees about utility of the code, safety of the code, suitability of the business model, investment advice, endorsement of the platform or its products, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

The information presented in this report is confidential and privileged. If you are reading this report, you agree to keep it confidential, not to copy, disclose or disseminate without the agreement of the client. If you are not the intended receptor of this document, remember that any disclosure, copying or dissemination of it is forbidden.

Version / Date	Description
0.1 (05.01.2026)	Layout
0.4 (10.01.2026)	Automated Security Testing / Manual Security Testing
0.5 (12.01.2026)	Verify Claims
0.9 (16.01.2026)	Summary and Recommendation
1.0 (28.01.2026)	Re-check after mitigation
1.1 (03.02.2026)	Final document (first audit)
1.2 (03.05.2026)	Re-check after mitigation (round 2) and final document
1.3 (23.05.2026)	New Scope Settlement Contracts
1.4 (01.06.2026)	Final Review

2. About the Project and Company

Company: Lambdaplex Labs LLC

Address: 5440 Wade Park Blvd, Suite 300, Raleigh, NC 27607 USA

Website: https://www.lambdaplex.io

Twitter (X): https://x.com/lambdaplex

GitHub: https://github.com/lambdaplexlabs

Instagram: https://instagram.com/lambdaplex

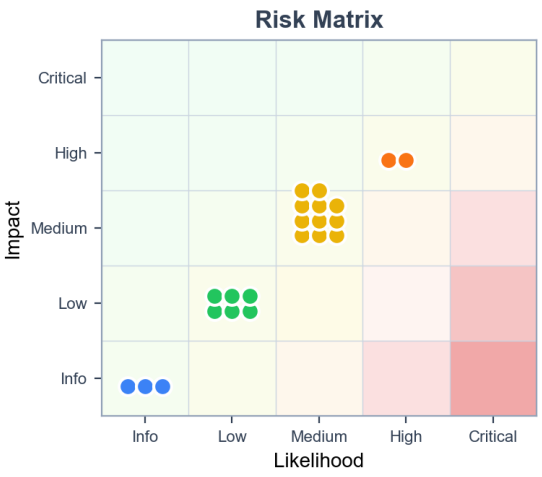
Lambdaplex is a Hedera-native, on-chain trading protocol that enables high-performance, non-custodial spot trading with advanced order functionality directly at the account level. Built on Hedera's Hiero Hooks and the Hedera Token Service, it embeds trading intent and execution rules into user accounts rather than centralized smart contracts. Users install a lightweight on-chain hook that governs how assets in their account may be exchanged via limit, market, stop-loss and take-profit orders, while custody always remains with the user. Order authorization is separated from order matching, so any broker, solver or matching engine may execute open orders, with the hook independently validating each transfer against the user's price limits, slippage bounds and oracle-based triggers. By leveraging Hedera's throughput and low, predictable fees, Lambdaplex targets professional and institutional trading use cases that require deterministic execution and strong self-custody guarantees.

The settlement layer extends this architecture with the Lambdaplex Settlements contract, which performs the final on-chain trade execution. It installs and tracks signed user orders, supports partial, full, market-style and stop/oracle-triggered fills, and accepts both signed retail orders and contract-style settlement authorizations using one-time settlement keys. For each batch it validates transfer assets, debit-approval flags, fees and order-lifecycle state, then moves assets through the Hedera Token Service via `cryptoTransfer` with replay protection on signed orders and settlement hashes. Together with the balanced vault and order hook, the settlement contract forms the third pillar of the Lambdaplex Core Contracts under audit.

3. Vulnerability & Risk Level

Risk represents the probability that a certain source-threat will exploit vulnerability, and the impact of that event on the organization or system. Risk Level is computed based on CVSS version 3.1.

Level	CVSS Score	Vulnerability	Required Action
CRITICAL	9.0 – 10.0	A vulnerability that can disrupt the contract functioning in a number of scenarios, or creates a risk that the contract may be broken.	Immediate action to reduce risk level.
HIGH	7.0 – 8.9	A vulnerability that affects the desired outcome when using a contract, or provides the opportunity to use a contract in an unintended way.	Implementation of corrective actions as soon as possible.
MEDIUM	4.0 – 6.9	A vulnerability that could affect the desired outcome of executing the contract in a specific scenario.	Implementation of corrective actions in a certain period.
LOW	0.1 – 3.9	A vulnerability that does not have a significant impact on possible scenarios for the use of the contract and is probably subjective.	Implementation of certain corrective actions or accepting the risk.
INFORMATIONAL	0.0	A vulnerability that has informational character but is not affecting any of the code.	An observation that does not determine a level of risk.



4. Auditing Strategy and Techniques Applied

Throughout the review process, care was taken to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

The auditing process follows a routine series of steps:

Code review that includes the following:

- Review of the specifications, sources, and instructions provided to softstack to make sure we understand the size, scope, and functionality of the smart contract.
- Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
- Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to softstack describe.

Testing and automated analysis that includes the following:

- Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
- Symbolic execution, which is analysing a program to determine what inputs causes each part of a program to execute.

Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarify, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.

Specific, itemized, actionable recommendations to help you take steps to secure your smart contracts.

5. Metrics

The metrics section gives the reader an overview of the size, quality, flows and capabilities of the codebase.

5.1 Tested Contract Files

Source repositories and commits for each contract in scope:

Contract	Source	Commit
Vault	https://github.com/lambdaplexlabs/lambdaplex-balanced-vault	`63887ccebad46e0cfac87cb6a7f6f8c77e3cf737`
Order Hook	https://github.com/lambdaplexlabs/lambdaplex-order-hook	`5124724871b15e147286c77da17cca57cba25ac6`
Settlement	https://github.com/lambdaplexlabs/lambdaplex-settlement	`ba3a346dc0fbb1789a297bc3089dac3c5a28da2b`

lambdaplex-balanced-vault-master

File	Fingerprint (MD5)
./lambdaplex-balanced-vault-master/contracts/base/AirdropDistributor.sol	`97692e564b9bfccd72202b4ff2408323`
./lambdaplex-balanced-vault-master/contracts/base/PLEXPairVault.sol	`58ced6a77f54817ba84e11d6d66d7be2`
./lambdaplex-balanced-vault-master/contracts/external/PLEXBrokerRegistry.sol	`68f9364fc035396b92e99c4a773ffe35`
./lambdaplex-balanced-vault-master/contracts/external/SupraRegistry.sol	`94b6a1ca576462a6e396f992dbf79de4`
./lambdaplex-balanced-vault-master/contracts/hook/PLEXVaultHook.sol	`9ea12b0670fde5d544adc9da3171d422`
./lambdaplex-balanced-vault-master/contracts/interfaces/IAirdropDistributor.sol	`a2ad4504364089b394233fe9ee44e258`
./lambdaplex-balanced-vault-master/contracts/interfaces/IERC20.sol	`b51b73ca61dee3132fbfe87a1bd0c282`
	`fb996833cc8513b10b2bbc25631e1e56`

File	Fingerprint (MD5)
./lambdaplex-balanced-vault-master/contracts/interfaces/IHederaTokenService.sol	
./lambdaplex-balanced-vault-master/contracts/interfaces/IHieroAccountAllowanceHook.sol	f54166fb728d0a9bff9bd467f68a5932
./lambdaplex-balanced-vault-master/contracts/interfaces/IHieroHook.sol	501418ad835858df20ba44945d44d2e5
./lambdaplex-balanced-vault-master/contracts/interfaces/IOrderFlowAllowance.sol	2ff1f5f82ea896af7540e4fa64f6b71a
./lambdaplex-balanced-vault-master/contracts/interfaces/IPLEXBrokerRegistry.sol	813794f7465ec6ae9142d9e3aaded5d7
./lambdaplex-balanced-vault-master/contracts/interfaces/ISupraRegistry.sol	aaf3e10ad96f995f4e6da1ddd2bfc31d
./lambdaplex-balanced-vault-master/contracts/libraries/DetailDecoder.sol	eefaaf88661f865d994f11aa1cae717b
./lambdaplex-balanced-vault-master/contracts/libraries/IocFokWriteback.sol	1ff12b4d3d0ef1adcbe68158b5dd95c9
./lambdaplex-balanced-vault-master/contracts/libraries/OrderType.sol	85a3f207c086a630f345ab01fb28e9d0
./lambdaplex-balanced-vault-master/contracts/libraries/PRBMathCommon.sol	06c6beda123ad8f98b859f02fd77970b
./lambdaplex-balanced-vault-master/contracts/libraries/PrefixDecoder.sol	bec5b119c6f9d0cb6c9bfaa19af15934
./lambdaplex-balanced-vault-master/contracts/libraries/SafeERC20.sol	4d866575c4d4f6569c263c696016d5aa
./lambdaplex-balanced-vault-master/contracts/PLEXPairVaultHedera.sol	4d4e561820da6a0fa6b6fc476c6ea458

lambdaplex-order-hook-master

File	Fingerprint (MD5)
./lambdaplex-order-hook-master/contracts/interfaces/IHieroAccountAllowanceHook.sol	f54166fb728d0a9bff9bd467f68a5932
./lambdaplex-order-hook-master/contracts/interfaces/IHieroHook.sol	501418ad835858df20ba44945d44d2e5
	2ff1f5f82ea896af7540e4fa64f6b71a

File	Fingerprint (MD5)
./lambdaplex-order-hook-master/contracts/interfaces/IOrderFlowAllowance.sol	
./lambdaplex-order-hook-master/contracts/interfaces/ISupraRegistry.sol	aaf3e10ad96f995f4e6da1ddd2bfc31d
./lambdaplex-order-hook-master/contracts/LambdaplexOrderHook.sol	99b972c961d06cf61994b6ccd6d4f97b
./lambdaplex-order-hook-master/contracts/libraries/DetailDecoder.sol	eefaaf88661f865d994f11aa1cae717b
./lambdaplex-order-hook-master/contracts/libraries/IocFokWriteback.sol	1ff12b4d3d0ef1adcbe68158b5dd95c9
./lambdaplex-order-hook-master/contracts/libraries/OrderType.sol	85a3f207c086a630f345ab01fb28e9d0
./lambdaplex-order-hook-master/contracts/libraries/PRBMathCommon.sol	06c6beda123ad8f98b859f02fd77970b
./lambdaplex-order-hook-master/contracts/libraries/PrefixDecoder.sol	bec5b119c6f9d0cb6c9bfaa19af15934
./lambdaplex-order-hook-master/contracts/oracle/SupraRegistry.sol	94b6a1ca576462a6e396f992dbf79de4

lambdaplex-settlement-master

File	Fingerprint (MD5)
./lambdaplex-settlement-master/contracts/interfaces/IHederaAccountService.sol	0b2258196ac9da5d84774e46fcbc2da6
./lambdaplex-settlement-master/contracts/interfaces/IHederaTokenService.sol	adde7ef3328a6b8097709a81311a750d
./lambdaplex-settlement-master/contracts/interfaces/IHieroAccountAllowanceHook.sol	f54166fb728d0a9bff9bd467f68a5932
./lambdaplex-settlement-master/contracts/interfaces/IHieroHook.sol	501418ad835858df20ba44945d44d2e5
./lambdaplex-settlement-master/contracts/interfaces/IOrderFlowAllowance.sol	2ff1f5f82ea896af7540e4fa64f6b71a
./lambdaplex-settlement-master/contracts/interfaces/ISupraRegistry.sol	be3654fb40fe4dca18e95b6c08679d84
./lambdaplex-settlement-master/contracts/interfaces/IVaultManager.sol	e3d8b287884764d57195473136dbba79

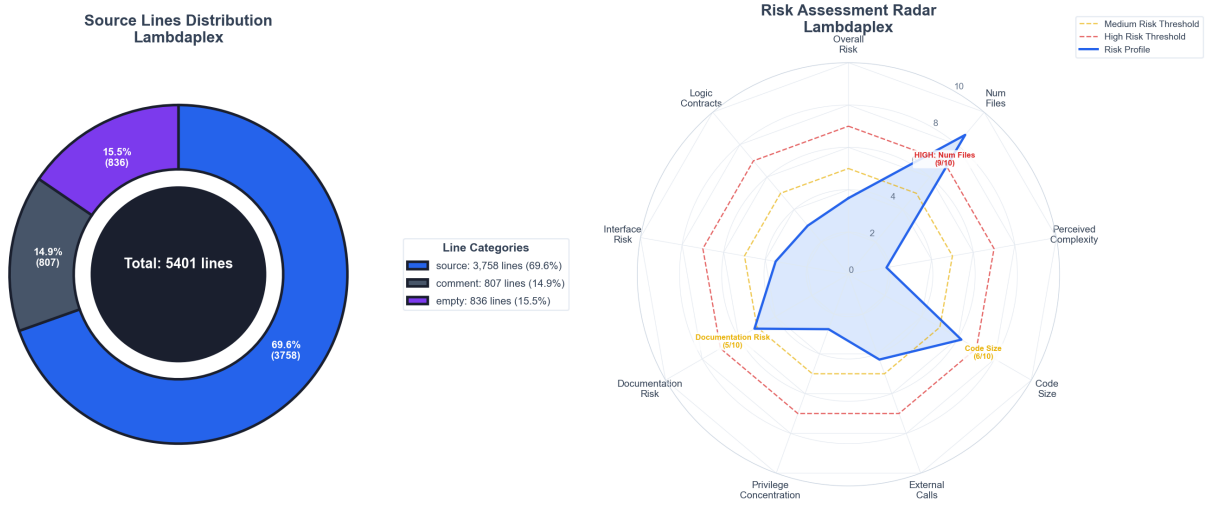
File	Fingerprint (MD5)
./lambdaplex-settlement-master/contracts/LambdaplexSettlements.sol	e89c44ed2b84031a29e1ea1e1dde1416
./lambdaplex-settlement-master/contracts/libraries/DecimalStrings.sol	8870c0cfc79a6570617380986c6f3058
./lambdaplex-settlement-master/contracts/libraries/DetailDecoder.sol	eefaaf88661f865d994f11aa1cae717b
./lambdaplex-settlement-master/contracts/libraries/OrderType.sol	85a3f207c086a630f345ab01fb28e9d0
./lambdaplex-settlement-master/contracts/libraries/PRBMathCommon.sol	06c6beda123ad8f98b859f02fd77970b
./lambdaplex-settlement-master/contracts/libraries/PrefixDecoder.sol	40ea653812beddee3d0b566f60b973fb
./lambdaplex-settlement-master/contracts/libraries/SupraTimestamp.sol	ce118b90202d1e8c4e75b8758439ab9f
./lambdaplex-settlement-master/contracts/oracle/SupraRegistry.sol	b1fdf92ec24144fed56336ad9e881845

lambdaplex-vault-main

File	Fingerprint (MD5)
./lambdaplex-vault-main/contracts/base/AirdropDistributor.sol	368e7242aaab9af5ba86872725949609
./lambdaplex-vault-main/contracts/base/PLEXPairVault.sol	a467f5a654dfc6fa1a5ee1ed8978c200
./lambdaplex-vault-main/contracts/external/PLEXBrokerRegistry.sol	68f9364fc035396b92e99c4a773ffe35
./lambdaplex-vault-main/contracts/external/SupraRegistry.sol	94b6a1ca576462a6e396f992dbf79de4
./lambdaplex-vault-main/contracts/hook/PLEXVaultHook.sol	7f6181cdf254764b5c2c1c8cb1fa415d
./lambdaplex-vault-main/contracts/interfaces/IAirdropDistributor.sol	a2ad4504364089b394233fe9ee44e258
./lambdaplex-vault-main/contracts/interfaces/IERC20.sol	b51b73ca61dee3132fbfe87a1bd0c282
./lambdaplex-vault-main/contracts/interfaces/IHederaTokenService.sol	fb996833cc8513b10b2bbc25631e1e56
./lambdaplex-vault-main/contracts/interfaces/IHieroAccountAllowanceHook.sol	f54166fb728d0a9bff9bd467f68a5932
./lambdaplex-vault-main/contracts/interfaces/IHieroHook.sol	501418ad835858df20ba44945d44d2e5
	2ff1f5f82ea896af7540e4fa64f6b71a

File	Fingerprint (MD5)
./lambdaplex-vault-main/contracts/interfaces/IOrderFlowAllowance.sol	
./lambdaplex-vault-main/contracts/interfaces/IPLEXBrokerRegistry.sol	813794f7465ec6ae9142d9e3aaded5d7
./lambdaplex-vault-main/contracts/interfaces/ISupraRegistry.sol	aaf3e10ad96f995f4e6da1ddd2bfc31d
./lambdaplex-vault-main/contracts/libraries/Decoder.sol	03c06452ae78e84157196906f4467eb6
./lambdaplex-vault-main/contracts/libraries/DetailDecoder.sol	eefaaf88661f865d994f11aa1cae717b
./lambdaplex-vault-main/contracts/libraries/IocFokWriteback.sol	ce3bfb9ae82e803d6583ed1bad78a0c6
./lambdaplex-vault-main/contracts/libraries/OrderType.sol	85a3f207c086a630f345ab01fb28e9d0
./lambdaplex-vault-main/contracts/libraries/PRBMathCommon.sol	06c6beda123ad8f98b859f02fd77970b
./lambdaplex-vault-main/contracts/libraries/PrefixDecoder.sol	bec5b119c6f9d0cb6c9bfaa19af15934
./lambdaplex-vault-main/contracts/PLEXPairVaultHedera.sol	dee27f6260891544f7f8ca16dd85127a

5.2 Source Lines & Risk



5.3 Capabilities

Module	Functions	Public	Restricted	Key Roles
AirdropDistributor	7	4	3	owner

Module	Functions	Public	Restricted	Key Roles
IAirdropDistributor	3	3	0	-
IERC20	3	3	0	-
IHederaAccountService	1	1	0	-
IHederaTokenService	1	1	0	-
IHieroAccountAllowanceHook	3	3	0	-
IOrderFlowAllowance	3	3	0	-
IPLEXBrokerRegistry	2	2	0	-
ISupraRegistry	12	12	0	-
IVaultManager	1	1	0	-
LambdaplexOrderHook	2	2	0	-
LambdaplexSettlements	13	11	2	owner
PLEXBrokerRegistry	2	1	1	owner
PLEXPairVault	19	15	4	owner
PLEXVaultHook	2	2	0	-
SupraRegistry	17	11	6	owner

5.4 Dependencies / External Imports

Import Path	Version
@openzeppelin/contracts/access/Ownable.sol	4.6.0
@openzeppelin/contracts/token/ERC20/ERC20.sol	4.6.0
@openzeppelin/contracts/token/ERC20/IERC20.sol	4.6.0
@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol	4.6.0
@openzeppelin/contracts/utils/math/SafeCast.sol	4.6.0
@openzeppelin/contracts/access/Ownable.sol	4.6.0
@openzeppelin/contracts/access/Ownable.sol	5.3.0
@openzeppelin/contracts/utils/Base64.sol	5.3.0
@openzeppelin/contracts/utils/ReentrancyGuard.sol	5.3.0

5.5 Source Units in Scope

lambdaplex-balanced-vault-master

File	Instructions	Functions	Lines	nSLOC	Comments
PLEXPairVaultHedera.sol	0	0	62	57	1
PrefixDecoder.sol	0	7	45	24	10
SafeERC20.sol	0	2	31	29	1
DetailDecoder.sol	0	6	42	27	9
PRBMathCommon.sol	0	2	133	62	52
IocFokWriteback.sol	0	2	61	46	8
OrderType.sol	0	4	27	21	1
SupraRegistry.sol	5	6	49	38	1
PLEXBrokerRegistry.sol	2	2	22	15	1
PLEXVaultHook.sol	0	8	403	294	52
PLEXPairVault.sol	10	44	1431	1025	201
AirdropDistributor.sol	6	7	111	79	10
IERC20.sol	3	3	8	6	1
IAirdropDistributor.sol	3	3	14	6	5
ISupraRegistry.sol	4	5	42	30	7
IPLEXBrokerRegistry.sol	2	2	8	5	1
IOrderFlowAllowance.sol	0	1	20	9	8
IHieroAccountAllowanceHook.sol	0	1	65	31	27
IHederaTokenService.sol	0	0	67	23	32
IHieroHook.sol	0	0	20	11	8

lambdaplex-order-hook-master

File	Instructions	Functions	Lines	nSLOC	Comments
LambdaplexOrderHook.sol	0	8	394	287	52
SupraRegistry.sol	5	6	49	38	1

File	Instructions	Functions	Lines	nSLOC	Comments
PrefixDecoder.sol	0	7	45	24	10
DetailDecoder.sol	0	6	42	27	9
PRBMathCommon.sol	0	2	133	62	52
IocFokWriteback.sol	0	2	61	46	8
OrderType.sol	0	4	27	21	1
ISupraRegistry.sol	4	5	42	30	7
IOrderFlowAllowance.sol	0	1	20	9	8
IHieroAccountAllowanceHook.sol	0	1	65	31	27
IHieroHook.sol	0	0	20	11	8

lambdaplex-settlement-master

File	Instructions	Functions	Lines	nSLOC	Comments
LambdaplexSettlements.sol	2	32	1303	1027	43
SupraRegistry.sol	4	5	45	35	1
PrefixDecoder.sol	0	7	45	24	10
DetailDecoder.sol	0	6	42	27	9
PRBMathCommon.sol	0	2	133	62	52
DecimalStrings.sol	0	1	25	21	1
OrderType.sol	0	4	27	21	1
SupraTimestamp.sol	0	1	15	10	3
ISupraRegistry.sol	2	2	23	18	1
IOrderFlowAllowance.sol	0	1	20	9	8
IHieroAccountAllowanceHook.sol	0	1	65	31	27
IHederaAccountService.sol	0	1	10	8	1
IHederaTokenService.sol	0	1	60	26	22
IVaultManager.sol	1	1	9	4	1
IHieroHook.sol	0	0	20	11	8

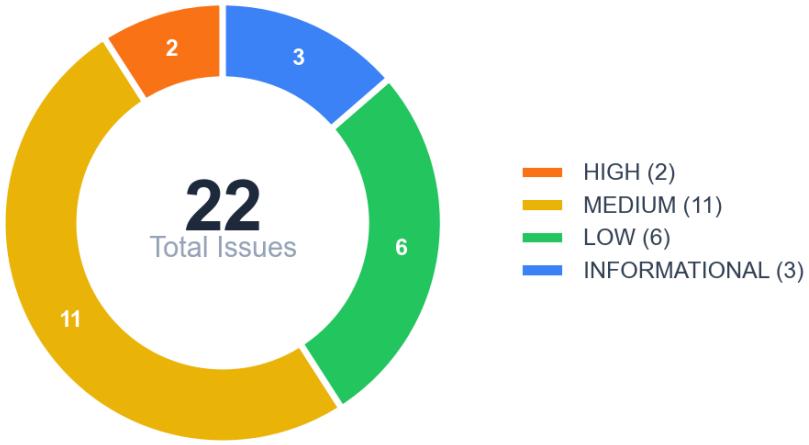
Total (all contracts): 53 instructions, 212 functions, 5401 lines, 3758 nSLOC

6. Scope of Work

This audit covers the Lambdaplex core contracts deployed on Hedera. The scope spans three codebases: the balanced vault (PLEXPairVault, PLEXVaultHook, AirdropDistributor, PLEXBrokerRegistry, SupraRegistry and supporting libraries), the order hook (LambdaplexOrderHook with its prefix/detail decoders, IOC/FOK writeback and Supra oracle integration), and the settlement contract (LambdaplexSettlements with its SupraRegistry, order-detail libraries and HTS cryptoTransfer integration). The audit focuses on validating share accounting, oracle integration, inventory rebalancing, reentrancy protection, reward streaming, signed-order authorization, settlement accounting and general security of privileged operations.

6.1 Findings Overview

Findings are classified by severity: Critical, High, Medium, Low, and Informational.



No	Title	Severity	Status
6.2.1	First-Depositor Vault Inflation Attack	HIGH	FIXED
6.2.2	Missing Zero-Price Protection in Oracle Factor	HIGH	FIXED
6.2.3	60-Second Staleness Window Vulnerability	MEDIUM	FIXED
6.2.4	Oracle Proof Replay Within Staleness Window	MEDIUM	FIXED

No	Title	Severity	Status
6.2.5	Eligible Shares Can Be Zero During Normal Operation	MEDIUM	ACKNOWLEDGED
6.2.6	minFill Validation Uses Executed Quantity, Not Total Order Quantity	MEDIUM	FIXED
6.2.7	Reward Token Array Unbounded Growth	MEDIUM	FIXED
6.2.8	Owner Fee Share Redemption Race Condition	MEDIUM	FIXED
6.2.9	lockupSecs and feeChangeDelaySecs Can Be Set to Zero, Disabling Core Protections	MEDIUM	FIXED
6.2.10	AirdropDistributor.SafeERC20 Still Uses High-Level transfer()/transferFrom(), Inconsistent With Vault's Fix	MEDIUM	FIXED
6.2.11	Mutable Oracle Pair Metadata Can Brick or Reprice Live Stop Orders	MEDIUM	ACKNOWLEDGED
6.2.12	Market Orders Can Set deviation == BIPS and Collapse Required Credit	MEDIUM	ACKNOWLEDGED
6.2.13	Signed Malformed Orders Can Persist and Revert Later Execution	MEDIUM	FIXED
6.2.14	Reward Carry Truncation (uint128)	LOW	FIXED
6.2.15	Unregistered Reward Token Can Block Claiming	LOW	FIXED
6.2.16	HTS associateToken Return Value Misinterpreted	LOW	FIXED
6.2.17	Missing Upper Bounds on uint64 Immutables Can Permanently DoS the Vault	LOW	FIXED
6.2.18	STALE_PRICE Tightened to 30 Seconds Without Complementary Round Validation — Availability Risk	LOW	ACKNOWLEDGED
6.2.19	claimRewards(address) Does Not Use try/catch — Single Bad Token Still Reverts That Path	LOW	FIXED
6.2.20	ownerRedeemFees Silently Caps feeSharesToBurn to Available Balance	INFORMATIONAL	FIXED
6.2.21	receive() Allows Anyone to Donate HBAR, Inflating Vault TVL	INFORMATIONAL	ACKNOWLEDGED
6.2.22	Dual SafeERC20 Library Definitions Create Maintenance Risk	INFORMATIONAL	ACKNOWLEDGED

6.2 Manual and Automated Vulnerability Test

6.2.1 First-Depositor Vault Inflation Attack

HIGH FIXED

DESCRIPTION

The PLEXPairVault contract is vulnerable to a classic first-depositor inflation attack (also known as "donation attack" or "share price manipulation"). The vulnerability exists because:

1. When totalShares == 0, the first depositor receives shares equal to their deposit value (1:1 ratio)

2. An attacker can directly transfer tokens to the vault via ERC20.transfer(), which increases totalAssets without minting new shares

3. Subsequent depositors receive shares calculated as (deposit * totalShares) / totalAssets, which rounds down due to integer division

4. When totalAssets is artificially inflated, victims receive disproportionately fewer shares or zero shares

The contract lacks any mitigation against this well-known attack vector: no virtual shares/assets offset (OpenZeppelin's recommended mitigation), no dead shares minting (Uniswap V2's mitigation), and no minimum first deposit requirement.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L916-920:

```
// Mint shares
uint256 sh = (totalShares == 0 || tvlQBefore == 0)
    ? principalQ
    : (principalQ * totalShares) / tvlQBefore;
require(sh > 0, "shares=0");
require(sh <= type(uint96).max, "shares overflow");
```

RECOMMENDATION

Implement OpenZeppelin's recommended virtual shares/assets offset to neutralise the attack:

Alternative mitigations include minting dead shares to address(0) on the first deposit (Uniswap V2 style), tracking internal balances rather than reading vault token balances, or requiring a substantial minimum first deposit.

```
uint256 private constant VIRTUAL_SHARES = 1e3;
uint256 private constant VIRTUAL_ASSETS = 1;

function _convertToShares(uint256 assets) internal view returns (uint256) {
    return (assets * (totalShares + VIRTUAL_SHARES)) / (_totalAssets() +
```

```
VIRTUAL_ASSETS);
}
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/634e3e3994395daab081c3a8c35f631a2fd1c907

6.2.2 Missing Zero-Price Protection in Oracle Factor

HIGH FIXED

DESCRIPTION

The `_oracleFactor()` function in both `LambdaplexOrderHook.sol` and `PLEXVaultHook.sol` does not validate that the oracle price returned from Supra is non-zero before using it for trigger price comparisons. When the oracle returns a zero price (due to oracle failure, manipulation, or feed issues), the trigger condition logic becomes exploitable or causes denial of service.

The vulnerability exists because `pi.prices[0]` from the oracle is directly used to calculate `oraclePrice` without any validation. With `oraclePrice == 0`, every `STOP_LT`-style check passes trivially (`0 <= triggerPrice`), allowing an attacker to trigger stop-loss orders when current market price wouldn't justify it. Conversely, every `STOP_GT`-style check fails (`0 >= positive` is false), causing denial of service for take-profit orders.

The sister contract `balanced-vault-master/contracts/base/PLEXPairVault.sol` correctly implements this check at line 327: `require(rawPrice > 0, "oracle: price=0");` — the protection is missing in the hook contracts.

CODE LOCATION

order-hook-master/contracts/LambdaplexOrderHook.sol & balanced-vault-master/contracts/hook/PLEXVaultHook.sol — `_oracleFactor()`:

```
function _oracleFactor(
    uint8 ot,
    uint256 d,
    address inToken,
    address outToken,
    bytes memory proof
) private returns (uint256 factorBps) {
    ISupraRegistry.PriceInfo memory pi = supra.verifyOracleProofV2(proof);
    require(pi.pairs.length == 1, "oracle: pair length");
    // ... staleness/pair checks ...

    // @audit-issue VULNERABILITY: pi.prices[0] can be 0, no validation performed
    uint256 oraclePrice = PRBMathCommon.mulDiv(pi.prices[0], scaleB, scaleA);
```

```
    // ...
}
```

RECOMMENDATION

Add a zero-price validation immediately after fetching the price proof, consistent with `PLEXPairVault.sol`. Apply the fix in both `order-hook-master/contracts/LambdaplexOrderHook.sol` and `balanced-vault-master/contracts/hook/PLEXVaultHook.sol`.

```
ISupraRegistry.PriceInfo memory pi = supra.verifyOracleProofV2(proof);
require(pi.pairs.length == 1, "oracle: pair length");

// Add this check - consistent with PLEXPairVault.sol line 327
require(pi.prices[0] > 0, "oracle: price=0");
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-order-hook/pull/1/commits/824ab8df822be96bb2793c361618ea314ff5138f

6.2.3 60-Second Staleness Window Vulnerability

MEDIUM FIXED

DESCRIPTION

The PLEXPairVault contract uses a 60-second staleness threshold (`STALE_PRICE = 60`) for validating oracle price proofs from Supra Oracle. This window allows oracle proofs up to 60 seconds old to be accepted for deposit and withdrawal operations.

During periods of high market volatility, cryptocurrency prices can move 5-10% within 60 seconds. An attacker with knowledge of real-time market prices could exploit stale oracle proofs to gain an unfair advantage in share calculations. The 60-second threshold is at the higher end of common DeFi practice for protocols handling deposits/withdrawals where price accuracy directly affects share calculations.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L64, L307-311:

```
uint256 constant STALE_PRICE = 60;

// --- Freshness checks ---
uint64 t = uint64(info.timestamp[0]);
uint64 nowU = uint64(block.timestamp);
require(t != 0, "oracle: ts=0");
```

```
require(t <= nowU, "oracle: future");
require(nowU - t <= STALE_PRICE, "oracle: stale");
```

RECOMMENDATION

Reduce `STALE_PRICE` from 60 to 20-30 seconds to align with industry best practices, and consider making it a configurable parameter bounded by sane min/max values:

For additional defence in depth, add a price-deviation check that compares the new oracle price against the most recent observation and reverts if it exceeds a configured threshold (e.g. 5%).

```
uint256 constant STALE_PRICE = 20;
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/6b00d7916b4235d9ad2d6749d5a12fc125a169fd

6.2.4 Oracle Proof Replay Within Staleness Window

MEDIUM FIXED

DESCRIPTION

The `LambdaplexOrderHook` contract verifies oracle proofs from Supra Oracle using the `_oracleFactor()` function. While it correctly validates that the proof timestamp is within the staleness window, it does not validate the proof's round number or track previously used proofs.

The Supra Oracle returns a `PriceInfo` struct containing a `round` field specifically designed to prevent replay attacks and ensure proof uniqueness. However, this field is completely ignored by the consuming contract. An attacker can:

1. Re-use an old proof (still within staleness) whose price is favourable, even after the current market price has moved.

2. Re-use the same proof to consume multiple stop orders across different users within the same window.

CODE LOCATION

order-hook-master/contracts/LambdaplexOrderHook.sol L140-210 — `_oracleFactor()`:

```
function _oracleFactor(
    uint8 ot,
    uint256 d,
    address inToken,
    address outToken,
    bytes memory proof
) private returns (uint256 factorBps) {
```

```
    ISupraRegistry.PriceInfo memory pi = supra.verifyOracleProofV2(proof);
    require(pi.pairs.length == 1, "oracle: pair length");
    require(
        pi.timestamp[0] <= block.timestamp &&
        pi.timestamp[0] >= block.timestamp - STALE_PRICE,
        "oracle: stale"
    );
    // NOTE: pi.round[0] is NEVER validated or tracked
}
```

RECOMMENDATION

Validate the round number and require monotonically increasing rounds per pair, and combine that with a tighter staleness window:

Alternatively, track used proof hashes in a `mapping(bytes32 => bool) usedProofs` to prevent exact replays.

```
uint256 constant STALE_PRICE = 30;
mapping(uint256 => uint256) public lastAcceptedRound;

require(
    pi.timestamp[0] <= block.timestamp &&
    pi.timestamp[0] >= block.timestamp - STALE_PRICE,
    "oracle: stale"
);

require(pi.round[0] >= lastAcceptedRound[pi.pairs[0]], "oracle: old round");
lastAcceptedRound[pi.pairs[0]] = pi.round[0];
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-order-hook/pull/1/commits/2da94c0c450e73304ffd4fe52c8557febad98576

6.2.5 Eligible Shares Can Be Zero During Normal Operation

MEDIUM ACKNOWLEDGED

DESCRIPTION

The `_eligibleShares()` function calculates shares eligible for rewards by excluding owner fee shares from the total. When all depositors withdraw their funds, only `ownerFeeShares` remain in the vault. In this state, `totalShares == ownerFeeShares`, causing `_eligibleShares()` to return 0.

When `_updateReward()` is called with zero eligible shares, rewards accumulate in `carry` instead of being distributed via `perShare`. Because owner fee shares are explicitly excluded from reward distribution, those rewards become permanently orphaned — new depositors after this period receive zero accrued rewards and `carry` can overflow `uint128` over extended periods.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L483-510:

```
function _eligibleShares() internal view returns (uint256) {
    uint256 ts = totalShares;
    uint256 fee = ownerFeeShares;
    return ts > fee ? (ts - fee) : 0;
}

function _updateReward(address rt) internal {
    // ...
    uint256 eligible = _eligibleShares();
    if (eligible == 0) {
        R.carry += uint128(R.rate * dt);  // Rewards trapped in carry
    } else {
        R.perShare += (R.rate * dt * 1e18) / eligible;
    }
    // ...
}
```

RECOMMENDATION

Block reward funding while no eligible shares exist, allow the owner to recover orphaned `carry`, or roll the orphaned `carry` into the next stream once eligible shares return:

At minimum, emit a dedicated event when rewards fall into `carry` so off-chain monitoring can detect the condition.

```
function onAirdropFunded(address token, uint256 amount) external onlyDistributor {
    require(_eligibleShares() > 0, "no eligible shares");
    // ... existing logic
}
```

6.2.6 minFill Validation Uses Executed Quantity, Not Total Order Quantity

MEDIUM FIXED

DESCRIPTION

The `minFill` validation in `LambdaplexOrderHook._applyOne()` validates the fill amount against the current remaining quantity instead of the original order quantity. When an order is partially filled, the remaining quantity is written back via `IocFokWriteback.update()`, but the `minFill` percentage in the order detail remains unchanged. Subsequent fills are validated against the reduced remaining quantity, causing the effective minimum fill requirement to grow as the order is consumed.

Example: a user places a 1000-unit order with `minFill = 50%`. After a 600-unit partial fill the remaining is 400 units; the next fill must now be ≥ 200 units (50% of 400) even though 75% of the original order has already been filled.

CODE LOCATION

order-hook-master/contracts/LambdaplexOrderHook.sol L250-259, libraries/IocFokWriteback.sol L41-48:

```
uint256 q    = d.quantity();      // This is REMAINING quantity, not original
uint256 mfb  = d.minFill();
uint256 take = q <= st.remaining ? q : st.remaining;

// per-order minFillBps: take / q >= mfb / 1e6
if (mfb > 0 && take != 0) {
    require(take * BIPS >= q * mfb, "min fill");  // Uses remaining 'q', not
original
}
```

RECOMMENDATION

Track the original order quantity and validate against it, or clear `minFill` after the first valid fill so the threshold only applies to opening a position:

```
bytes32 newDetail = DetailDecoder.encode(
    rem,
    detailWord.numerator(),
    detailWord.denominator(),
    detailWord.deviation(),
    0 // set minFill to zero after first valid fill
);
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-order-hook/pull/1/commits/82ce5efb7e0068336e28afd8419b298c466d327d

6.2.7 Reward Token Array Unbounded Growth

MEDIUM FIXED

DESCRIPTION

The `rewardTokens` array in `PLEXPairVault` grows unboundedly when new reward tokens are funded via the distributor. Each unique token funded is permanently added to the array with no mechanism to remove exhausted or deprecated tokens. The settlement and claim functions iterate over the entire array, making `claimAllRewards()` and `_settleRewards()` cost O(n) gas in the number of reward tokens. An attacker who can fund the distributor with worthless tokens can grief every depositor by inflating gas costs.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L129, L476-480, L515-537, L605-618:

```
address[] public rewardTokens; // keep small

function _ensureListedReward(address rt) internal {
    for (uint i = 0; i < rewardTokens.length; i++)
        if (rewardTokens[i] == rt) return;
    rewardTokens.push(rt);  // No MAX_REWARD_TOKENS check
}
```

RECOMMENDATION

Cap the array length, expose an admin function to remove fully-distributed tokens, switch to an O(1) `mapping` for the dedup check, and restrict who can fund tokens (or require a minimum funding amount):

Combine with a `removeDepletedRewardToken(uint256 index)` function that swaps and pops tokens after asserting `distributor.remaining(...) == 0` and no pending rate/carry.

```
uint256 constant MAX_REWARD_TOKENS = 20;
mapping(address => bool) public isRewardToken;

function _ensureListedReward(address rt) internal {
    if (isRewardToken[rt]) return;
    require(rewardTokens.length < MAX_REWARD_TOKENS, "too many reward tokens");
    isRewardToken[rt] = true;
    rewardTokens.push(rt);
}
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/cc45e0f840e5e1387406cd8639c0a5c43d607d6e

6.2.8 Owner Fee Share Redemption Race Condition

MEDIUM FIXED

DESCRIPTION

The `ownerRedeemFees()` function allows the vault owner to redeem accrued management fee shares for BASE/QUOTE tokens. The function lacks slippage protection parameters (`minBaseOut`, `minQuoteOut`), making the redemption subject to MEV attacks and front-running on EVM-compatible deployments.

The payout is calculated at execution time based on current TVL, current share proportions and current oracle price. Any deposit or withdrawal that occurs between the owner's transaction submission and execution will change the redemption value, potentially resulting in the owner receiving less (or more) than expected.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L1010-1063 — `ownerRedeemFees()`:

```
function ownerRedeemFees(
    uint256 feeSharesToBurn,
    bytes memory supraArgs
) external onlyOwner nonReentrant {
    // ...
    // Value owed (QUOTE terms) - No slippage protection
    uint256 valueOutQ = (tvlQ * burn) / tsBefore;

    (uint256 payBase, uint256 payQuote) = _payoutOverweightThenBalanced(
        valueOutQ, price, scale, baseBal, quoteBal, imb
    );

    // Burn fee-shares and shrink total supply
    ownerFeeShares = available - burn;
    totalShares = tsBefore - burn;

    // Payout to owner - No minimum output validation
    _transferOut(BASE, payable(owner()), payBase);
    _transferOut(QUOTE, payable(owner()), payQuote);

    emit OwnerFeeRedeemed(burn, payBase, payQuote);
}
```

RECOMMENDATION

Add explicit slippage protection parameters and validate the resulting payout before transferring:

Also expose a `previewOwnerRedeemFees(...)` view function and consider a `deadline` parameter.

```
function ownerRedeemFees(
    uint256 feeSharesToBurn,
    bytes memory supraArgs,
    uint256 minBaseOut,
    uint256 minQuoteOut
) external onlyOwner nonReentrant {
    // ... compute payBase / payQuote ...
    require(payBase >= minBaseOut, "slippage: base");
    require(payQuote >= minQuoteOut, "slippage: quote");
    // ...
}
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/e2e1b994024d5d71110e74121553820e6184e580

6.2.9 lockupSecs and feeChangeDelaySecs Can Be Set to Zero, Disabling Core Protections

MEDIUM FIXED

DESCRIPTION

The constructor validates `vestingSecs_ > 0` and `initialBalanceTolBips_ <= 50_000`, but does not validate `lockupSecs_` or `feeChangeDelaySecs_`. Both are `uint64` immutables set once at construction, and both are critical to security:

- `lockupSecs = 0` → deposits can be withdrawn in the same block, enabling flash-loan deposit + withdraw attack vectors that extract value from the rebalancing policy.

- `feeChangeDelaySecs = 0` → owner can change the management fee rate with zero delay, bypassing the timelock protection entirely. `pendingOwnerFeeTs = nowTs + 0 = nowTs`, so the pending fee is immediately effective. The cooldown check `nowTs >= lastFeeChangeTs + 0` also passes every call.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L220-248 (constructor):

```
// L231: vestingSecs is validated
require(vestingSecs_ > 0, "vesting=0");
// BUT lockupSecs_ and feeChangeDelaySecs_ are NOT validated
lockupSecs = lockupSecs_;                 // can be 0
feeChangeDelaySecs = feeChangeDelaySecs_; // can be 0
```

RECOMMENDATION

Add minimum bounds in the constructor:

```
require(lockupSecs_ > 0, "lockup=0");
require(feeChangeDelaySecs_ >= 1 days, "delay too short");
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/6/changes/0e6e77b1abd79d396bc14e9664a7390654571c54

6.2.10 AirdropDistributor.SafeERC20 Still Uses High-Level transfer()/ transferFrom(), Inconsistent With Vault's Fix

MEDIUM FIXED

DESCRIPTION

A previous change upgraded the vault's `SafeERC20` to use a low-level `call()` to support tokens that don't return a `bool`. However, the `AirdropDistributor` still defines its own separate `SafeERC20` library that uses the high-level pattern:

This means:

1. If a reward token doesn't return `bool` on `transfer()`, the Solidity compiler reverts on the ABI decode step → `claimTo()` always reverts for that token.

2. The vault's `claimAllRewards()` `try/catch` would catch this, but the user can never claim that reward via `claimRewards(rt)` directly (no `try/catch` there — see L-07).

3. `fund()`'s `safeTransferFrom` would also revert, preventing funding with such tokens.

4. The two `SafeERC20` libraries are semantically inconsistent — the vault accepts non-bool tokens, but the distributor rejects them.

CODE LOCATION

balanced-vault-master/contracts/base/AirdropDistributor.sol L22-29:

```
// AirdropDistributor.sol L23-28
library SafeERC20 {
    function safeTransfer(IERC20 t, address to, uint256 v) internal {
        bool ok = t.transfer(to, v); require(ok, "TRANSFER_FAILED");
    }
    function safeTransferFrom(IERC20 t, address from, address to, uint256 v)
internal {
        bool ok = t.transferFrom(from, to, v); require(ok,
"TRANSFER_FROM_FAILED");
```

```
        }
    }
```

RECOMMENDATION

Unify the `SafeERC20` implementation. Copy the vault's low-level `call()` pattern into the distributor, or extract it into a shared library imported by both contracts.

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/6/changes/d8eb291b0267db837c22545b5c277bb3fee822b1

6.2.11 Mutable Oracle Pair Metadata Can Brick or Reprice Live Stop Orders

MEDIUM ACKNOWLEDGED

DESCRIPTION

`registerPair()` overwrites `pairs[pairId]` without checking whether the pair already exists. Active stop orders that rely on that pair id can be silently bricked by changing the pair tokens, or repriced by changing the token decimal metadata.

The settlement contract later consumes that mutable pair metadata when validating oracle-triggered orders:

The same pair id can be overwritten after an order is installed. The later stop trigger evaluates the unchanged order bytes against changed pair metadata, causing either `oracle: wrong pair` or a changed trigger outcome. PoC tests demonstrate that a pair overwrite after order installation can brick a live stop order, and that decimal-metadata overwrite changes the trigger outcome without changing the order bytes.

CODE LOCATION

lambdaplex-settlement-master/contracts/oracle/SupraRegistry.sol L24-35 & lambdaplex-settlement-master/contracts/LambdaplexSettlements.sol L910-946:

```
function registerPair(
    uint256 pairId,
    address tokenA,
    address tokenB
) external onlyOwner() {
    require(tokenA != tokenB, 'IDENTICAL_ADDRESSES');
    pairs[pairId] = TokenPair(
        tokenA,
        tokenB,
        tokenA == address(0) ? 8 : IERC20(tokenA).decimals(),
```

```
        tokenB == address(0) ? 8 : IERC20(tokenB).decimals());
}
```

```
ISupraRegistry.TokenPair memory pair = supra.getPair(pi.pairs[0]);
require(
    (inToken == pair.tokenA && outToken == pair.tokenB) ||
    (inToken == pair.tokenB && outToken == pair.tokenA),
    "oracle: wrong pair"
);
uint256 oraclePrice;
uint256 execPrice;
{
    uint256 decimals = pi.decimal[0];
    require(decimals < 39, "oracle: decimals");
    uint256 da = uint256(pair.decimalsA);
    uint256 db = uint256(pair.decimalsB);
    require(da <= 38 && db <= 38, "oracle: token decimals");
    uint256 scaleA = 10 ** da;
    uint256 scaleB = 10 ** db;
    oraclePrice = PRBMathCommon.mulDiv(pi.prices[0], scaleB, scaleA);
}
```

RECOMMENDATION

Make pair ids immutable after registration, or require a timelocked pair migration flow that records old and new metadata and gives active order owners an exit window.

```
function registerPair(
    uint256 pairId,
    address tokenA,
    address tokenB
) external onlyOwner() {
    require(tokenA != tokenB, 'IDENTICAL_ADDRESSES');
    require(pairs[pairId].tokenA == address(0), 'pair already registered');
    pairs[pairId] = TokenPair(
        tokenA,
        tokenB,
        tokenA == address(0) ? 8 : IERC20(tokenA).decimals(),
        tokenB == address(0) ? 8 : IERC20(tokenB).decimals());
}
```

6.2.12 Market Orders Can Set deviation == BIPS and Collapse Required Credit

MEDIUM ACKNOWLEDGED

DESCRIPTION

For market-style oracle orders, `_oracleFactor()` permits `slip <= BIPS` and computes `factorBps = BIPS - slip`. If `slip == BIPS`, `factorBps` becomes zero. `_applyOne()` then multiplies the required credit by that zero factor, allowing settlement where the seller receives negligible or zero effective consideration.

A market order selling 100 TOKEN_A can settle while the seller receives only 1 TOKEN_B, because `deviation == BIPS` makes the required-credit factor zero. The PoC demonstrates a settlement event where the seller receives negligible credit relative to the debit.

CODE LOCATION

lambdaplex-settlement-master/contracts/LambdaplexSettlements.sol L958-966, L995-1003:

```
if (OrderType.isMarketStyle(ot)) {
    require(slip <= BIPS, "slip > 1e6");
    factorBps = BIPS - slip;
} else {
    factorBps = BIPS;
}
```

```
uint256 need_i = PRBMathCommon.mulDiv(
    d.numerator(),
    take,
    d.denominator()
);
if (factorBps != BIPS) {
    need_i = PRBMathCommon.mulDiv(need_i, factorBps, BIPS);
}
```

RECOMMENDATION

Require `deviation < BIPS` for market-style orders, and reject settlements where the computed required output is zero for a nonzero debit. Add regression tests for `BIPS - 1`, `BIPS`, and over-`BIPS` deviation values.

```
if (OrderType.isMarketStyle(ot)) {
    require(slip < BIPS, "slip >= 1e6");
    factorBps = BIPS - slip;
} else {
    factorBps = BIPS;
}
```

```
// ...
require(need_i > 0, "need=0");
```

6.2.13 Signed Malformed Orders Can Persist and Revert Later Execution

MEDIUM FIXED

DESCRIPTION

`_installSignedOrders()` persists any nonzero signed retail detail after signature validation and duplicate checks. It does not reject `denominator == 0`. The malformed order remains in `orders[owner][key]`, then later execution paths call `PRBMathCommon.mulDiv(..., d.denominator())` and revert. This creates durable poison-order state that can repeatedly revert stop triggering or mixed settlement batches until the stored order is separately neutralized.

The stored malformed detail is later consumed by unchecked `denominator` math:

A signed malformed order with `denominator == 0` installs successfully and remains in storage. Later trigger or settlement execution reverts, and the malformed order remains stored after the revert.

CODE LOCATION

lambdaplex-settlement-master/contracts/LambdaplexSettlements.sol L580-640:

```
function _installSignedOrders(
    Order[] calldata signedOrders,
    bytes32 settlementHash_
)
    private
    returns (
        address[] memory contractApprovals,
        uint256 contractApprovalCount
    )
{
    contractApprovals = new address[](signedOrders.length);
    for (uint256 i = 0; i < signedOrders.length; i++) {
        Order calldata order_ = signedOrders[i];
        if (order_.owner == address(0)) {
            continue;
        }
        if (order_.detail == bytes32(0)) {
            // ... contract-settlement authorization path ...
            continue;
        }
        if (
            installed[order_.owner][order_.key] ||
```

```
            orders[order_.owner][order_.key] != bytes32(0)
        ) {
            continue;
        }
        if (
            !_isOrderAuthorized(
                order_.owner,
                order_.key,
                order_.detail,
                order_.signature
            )
        ) {
            continue;
        }
        installed[order_.owner][order_.key] = true;
        orders[order_.owner][order_.key] = order_.detail;
        emit OrderInstalled(order_.owner, order_.key, order_.detail);
    }
}
```

```
uint256 nmr = d.numerator();
uint256 dnm = d.denominator();
execPrice = (inToken == pair.tokenA)
    ? PRBMathCommon.mulDiv(nmr, scale, dnm)
    : PRBMathCommon.mulDiv(dnm, scale, nmr);

uint256 need_i = PRBMathCommon.mulDiv(
    d.numerator(),
    take,
    d.denominator()
);
```

RECOMMENDATION

Validate detail math invariants before installing retail orders. At minimum reject `denominator == 0`, `numerator == 0` for paths that invert it, and out-of-range deviation/minFill values before setting `installed` or writing `orders`.

```
require(d.denominator() != 0, "bad detail: dnm=0");
require(d.numerator() != 0, "bad detail: nmr=0");
require(d.deviation() <= BIPS, "bad detail: dev");
installed[order_.owner][order_.key] = true;
orders[order_.owner][order_.key] = order_.detail;
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-settlement/commit/3fe6a1ebf093f5d1a707afc73f88595e5ea61ac8

6.2.14 Reward Carry Truncation (uint128)

LOW FIXED

DESCRIPTION

The `PLEXPairVault` contract uses explicit `uint128()` casts when storing reward carry values. In Solidity 0.8+, explicit type casts do not revert on overflow — they silently truncate the value to fit the smaller type. If `rate * dt` exceeds `type(uint128).max` (~3.4e38), the excess bits are discarded without any error, resulting in permanent loss of reward tokens.

The vulnerable pattern appears in three locations: `_updateReward` (L500) when no eligible shares exist, and `onAirdropFunded` (L566 active stream, L578 fresh stream) when storing the remainder.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L500, L566, L578:

```
if (eligible == 0) {
    R.carry += uint128(R.rate * dt);  // @audit LINE 500 - Truncation risk
} else {
    R.perShare += (R.rate * dt * 1e18) / eligible;
}

// L566 (active stream reconfigure)
R.carry = uint128(totalToStream - R.rate * remainingTime);

// L578 (fresh stream)
R.carry = uint128(totalToStream - R.rate * VESTING_SECS);
```

RECOMMENDATION

Replace explicit casts with OpenZeppelin's `SafeCast.toUint128()` so an overflow reverts, or widen `carry` to `uint256`:

```
import "@openzeppelin/contracts/utils/math/SafeCast.sol";

R.carry += SafeCast.toUint128(R.rate * dt);
R.carry  = SafeCast.toUint128(totalToStream - R.rate * remainingTime);
R.carry  = SafeCast.toUint128(totalToStream - R.rate * VESTING_SECS);
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/0b424d2586e3127d7b5bb0ea3c91a9b5a1453f6c

6.2.15 Unregistered Reward Token Can Block Claiming

LOW FIXED

DESCRIPTION

`claimAllRewards()` iterates through all registered reward tokens and calls `distributor.claimTo(...)` sequentially. If any single reward token transfer fails during this loop (token paused, blacklisted, distributor drained, or a non-bool-returning ERC20), the entire transaction reverts and users cannot claim any rewards — even those from healthy tokens. This is a classic DoS pattern where one failing component blocks the entire operation.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L605-618 — `claimAllRewards()`:

```
function claimAllRewards() external nonReentrant {
    require(address(distributor) != address(0), "distributor not set");
    _settleRewards(msg.sender);
    for (uint i = 0; i < rewardTokens.length; i++) {
        address rt = rewardTokens[i];
        UserReward storage U = userRewards[msg.sender][rt];
        uint256 amt = U.accrued;
        if (amt > 0) {
            U.accrued = 0;
            distributor.claimTo(rt, msg.sender, amt);  // Reverts here block
entire function
            emit RewardClaimed(rt, msg.sender, amt);
        }
    }
}
```

RECOMMENDATION

Wrap each per-token claim in a `try/catch` so a failing token does not block the others, and preserve `U.accrued` on failure so the user can retry later:

Also expose an admin `removeRewardToken(address)` for permanently broken tokens.

```
event RewardClaimFailed(address indexed rewardToken, address indexed user, uint256
amount);

for (uint i = 0; i < rewardTokens.length; i++) {
```

```
        address rt = rewardTokens[i];
        UserReward storage U = userRewards[msg.sender][rt];
        uint256 amt = U.accrued;
        if (amt > 0) {
            try distributor.claimTo(rt, msg.sender, amt) {
                U.accrued = 0;
                emit RewardClaimed(rt, msg.sender, amt);
            } catch {
                emit RewardClaimFailed(rt, msg.sender, amt);
                // Leave U.accrued intact for retry via claimRewards(rt)
            }
        }
    }
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/fde16db047ff8801c418b5d07a0c9065d8c7e5de

6.2.16 HTS associateToken Return Value Misinterpreted

LOW FIXED

DESCRIPTION

The `AirdropDistributor` contract uses low-level calls to the Hedera Token Service (HTS) precompile at address `0x167` for token association and disassociation. The implementation has several flaws in handling the return values:

1. Ignoring success boolean — the `.call()` returns `(bool success, bytes memory data)`, but the code discards `success` using `( , bytes memory result)`.

2. Empty result causes unclear revert — if the precompile call fails entirely, `result` is empty and `abi.decode` reverts with an unhelpful error.

3. Magic number `22` — the code hard-codes `22` instead of using a documented `HederaResponseCodes.SUCCESS` constant.

4. `TOKEN_ALREADY_ASSOCIATED` (194) not handled — re-associating an already-associated token returns code 194, which fails the `== 22` check even though it is not an error.

5. Function name typo — the code uses `disassociateToken` but the official HTS precompile function is `dissociateToken` (no extra `as`). The selector mismatch makes the call fail with an empty result.

CODE LOCATION

balanced-vault-master/contracts/base/AirdropDistributor.sol L98-101, L109-112:

```
function associateToken(address token) external onlyOwner() {
    ( , bytes memory result) = address(0x167).call(
        abi.encodeWithSignature("associateToken(address,address)", address(this),
token)
    );
    require (abi.decode(result, (int32)) == 22);
}

function dissociateToken(address token) external onlyOwner() {
    ( , bytes memory result) = address(0x167).call(
        abi.encodeWithSignature("disassociateToken(address,address)",
address(this), token)
    );
    require (abi.decode(result, (int32)) == 22);
    modifyAllowed(token, false);
}
```

RECOMMENDATION

Capture the success boolean, verify the response length, accept `TOKEN_ALREADY_ASSOCIATED_TO_ACCOUNT`, fix the function selector, and prefer using `HederaTokenService` / `HederaResponseCodes` from the official Hedera library:

```
int32 constant HTS_SUCCESS = 22;
int32 constant HTS_TOKEN_ALREADY_ASSOCIATED = 194;

function associateToken(address token) external onlyOwner() {
    (bool success, bytes memory result) = address(0x167).call(
        abi.encodeWithSignature("associateToken(address,address)", address(this),
token)
    );
    require(success, "HTS call failed");
    require(result.length >= 4, "Invalid HTS response");
    int32 responseCode = abi.decode(result, (int32));
    require(
        responseCode == HTS_SUCCESS || responseCode ==
HTS_TOKEN_ALREADY_ASSOCIATED,
        "HTS association failed"
    );
}
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/1/commits/855875c2ec9bc930bdb46d8465927d1ebc77fe6b

6.2.17 Missing Upper Bounds on uint64 Immutables Can Permanently DoS the Vault

LOW FIXED

DESCRIPTION

`lockupSecs`, `feeChangeDelaySecs` and `vestingSecs` are `uint64` immutables set once in the constructor with no upper bound validation. They participate in `uint64` additions that Solidity 0.8.x protects with built-in overflow checks — meaning an absurdly large value causes a revert, not a silent wrap.

A deployment with e.g. `lockupSecs = type(uint64).max` would cause every deposit to revert on the overflow check at `d.lockupUntil = nowU64 + lockupSecs;` (L1100), permanently bricking the vault. While this is a deployment-time configuration error rather than a runtime exploit, it is a silent footgun with no validation safeguard.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L220-248 (constructor), L277, L708, L1100:

```
d.lockupUntil = nowU64 + lockupSecs;          // L1100
pendingOwnerFeeTs = nowTs + feeChangeDelaySecs; // L277
R.periodFinish = nowU64 + vestingSecs;        // L708
```

RECOMMENDATION

Add upper-bound constructor validation:

```
require(lockupSecs_ <= 365 days, "lockup too long");
require(vestingSecs_ <= 365 days, "vesting too long");
require(feeChangeDelaySecs_ <= 365 days, "delay too long");
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/6/changes/ee5a14458b5d3ed92ef72e5c6b8193aac1cabd85

6.2.18 STALE_PRICE Tightened to 30 Seconds Without Complementary Round Validation — Availability Risk

LOW ACKNOWLEDGED

DESCRIPTION

The staleness window was correctly tightened from 60s to 30s as part of oracle hardening:

Reducing the window is the right direction — a shorter staleness window shrinks the exploitation surface for stale-price attacks. However, the tighter window was applied without the complementary round-number validation that would allow it to work safely under all conditions. Since the vault relies solely on the timestamp check (no round tracking), the 30-second window must single-handedly guarantee both security (reject stale prices) and availability (accept valid ones).

On Hedera, block times are ~3-5 seconds, but Supra oracle publication cadence is not guaranteed to stay under 30 seconds for all pairs at all times. If the feed cadence exceeds 30s during congestion or oracle maintenance, all deposits and withdrawals revert with `"oracle: stale"`, locking user funds until a fresh proof appears. The `emergencyMode` safety valve exists but is permanent and one-way.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L73:

```
uint256 constant STALE_PRICE = 30;
```

RECOMMENDATION

Preferred: add round-number validation so the timestamp check only handles clock-drift sanity:

Alternative: make `STALE_PRICE` a constructor parameter bounded to e.g. 30-300 s so deployments can tune it to the actual oracle cadence of their target pairs. Minimum: confirm with Supra that the publication cadence for all target pairs is consistently < 30 s under adverse conditions and document this operational dependency.

```
mapping(uint256 => uint256) public lastAcceptedRound;
require(pi.round[0] >= lastAcceptedRound[pair], "oracle: old round");
lastAcceptedRound[pair] = pi.round[0];
```

6.2.19 claimRewards(address) Does Not Use try/catch — Single Bad Token Still Reverts That Path

LOW FIXED

DESCRIPTION

`claimAllRewards()` was upgraded with `try/catch` (see L-02) but the single-token `claimRewards(address rewardToken)` was not wrapped in `try/catch`:

If a specific reward token is permanently broken (paused, blacklisted, or a non-bool-returning token per M-08), `claimRewards(rt)` will always revert for that token with no fallback path — the user must use `claimAllRewards()` to skip it. No funds are at risk because Solidity's atomic revert restores `U.accrued`, but the API is inconsistent.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L725-735:

```
function claimRewards(address rewardToken) external nonReentrant {
    require(address(distributor) != address(0), "distributor not set");
    _settleRewards(msg.sender);
    UserReward storage U = userRewards[msg.sender][rewardToken];
    uint256 amt = U.accrued;
    U.accrued = 0;
    if (amt > 0) {
        distributor.claimTo(rewardToken, msg.sender, amt);
        emit RewardClaimed(rewardToken, msg.sender, amt);
    }
}
```

RECOMMENDATION

Wrap the per-token call in `try/catch` for symmetry with `claimAllRewards()`, or document that `claimAllRewards()` is the resilient path for broken tokens:

```
try distributor.claimTo(rewardToken, msg.sender, amt) {
    emit RewardClaimed(rewardToken, msg.sender, amt);
} catch {
    U.accrued = amt; // restore
    emit RewardClaimFailed(rewardToken, msg.sender, amt);
}
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/6/changes/36554d99a9a3384734e6b2d2e067240fa6377c2b

6.2.20 ownerRedeemFees Silently Caps feeSharesToBurn to Available Balance

INFORMATIONAL FIXED

DESCRIPTION

If the owner requests more fee shares than available, the function silently caps to `available`. This could be surprising if the owner expected to burn exactly `feeSharesToBurn` shares — they would receive a different payout than expected, and the slippage parameters added by M-06's mitigation only protect against price drift, not against the silent quantity reduction.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L1199-1200:

```
uint256 burn = feeSharesToBurn > available ? available : feeSharesToBurn;
require(burn > 0, "no feeShares");
```

RECOMMENDATION

Make the behaviour explicit, either by reverting on excess or by clearly documenting the capping:

```
require(feeSharesToBurn <= available, "exceeds available");
```

MITIGATION

https://github.com/lambdaplexlabs/lambdaplex-balanced-vault/pull/6/changes/91af418de83c7eee62ef97daacfd02affd1f8743

6.2.21 receive() Allows Anyone to Donate HBAR, Inflating Vault TVL

INFORMATIONAL ACKNOWLEDGED

DESCRIPTION

The vault has an unrestricted `receive() external payable {}`. When `BASE` or `QUOTE` is HBAR (`address(0)`), anyone can send HBAR directly to the vault, inflating balance readings. This inflates TVL and can manipulate share pricing in favour of existing depositors (or against new ones).

The `VIRTUAL_SHARES`/`VIRTUAL_VALUEQ` offset added for H-01 mitigates first-depositor donation attacks, but direct HBAR donations after depositors exist still dilute new depositors' share calculations.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L1248:

RECOMMENDATION

Restrict `receive()` so it only accepts HBAR during `depositWithPolicy` execution (using a transient flag), or document the donation behaviour as accepted and rely on internal accounting rather than `address(this).balance` for share pricing.

6.2.22 Dual SafeERC20 Library Definitions Create Maintenance Risk

INFORMATIONAL ACKNOWLEDGED

DESCRIPTION

Two different `SafeERC20` library implementations exist in the codebase:

1. The vault's (low-level `call()`, handles non-bool-returning tokens).

2. The distributor's (high-level `transfer()`/`transferFrom()`, reverts on non-bool tokens).

This divergence creates a maintenance hazard where future developers might assume both behave identically. See M-08 for the functional impact.

CODE LOCATION

balanced-vault-master/contracts/base/PLEXPairVault.sol L23, balanced-vault-master/contracts/base/AirdropDistributor.sol L22:

RECOMMENDATION

Extract a single canonical `SafeERC20` into `libraries/SafeERC20.sol` (or import OpenZeppelin's `SafeERC20`) and use it from both `PLEXPairVault` and `AirdropDistributor` so the behaviour cannot drift.

7. Executive Summary

Two independent softstack experts performed an unbiased and isolated audit of the smart contract provided by the Lambdaplex team. The main objective of the audit was to verify the security and functionality claims of the smart contract. The audit process involved a thorough manual code review and automated security testing.

Overall, the audit identified a total of 22 issues, classified as follows:

- No critical issues were found
- 2 high severity issues were found
- 11 medium severity issues were found
- 6 low severity issues were found
- 3 informational severity issues were found

The Lambdaplex team has successfully addressed all identified issues from the audit. All vulnerabilities have been mitigated based on the recommendations provided in the report. A follow-up review confirms that the fixes have been implemented effectively, ensuring the security and functionality of the smart contract.

8. About the Auditor

Established in 2017 under the name Chainsulting, and rebranded as softstack GmbH in 2023, softstack has been a trusted name in Web3 Security space. Within the rapidly growing Web3 industry, softstack provides a comprehensive range of offerings that include software development, cybersecurity, and consulting services. Softstack's competency extends across the security landscape of prominent blockchains like Solana, Tezos, TON, Ethereum and Polygon. The company is widely recognized for conducting thorough code audits aimed at mitigating risk and promoting transparency.

The firm's proficiency lies particularly in assessing and fortifying smart contracts of leading DeFi projects, a testament to their commitment to maintaining the integrity of these innovative financial platforms. To date, softstack plays a crucial role in safeguarding over $100 billion worth of user funds in various DeFi protocols.

Underpinned by a team of industry veterans possessing robust technical knowledge in the Web3 domain, softstack offers industry-leading smart contract audit services. Committed to evolving with their clients' ever-changing business needs, softstack's approach is as dynamic and innovative as the industry it serves.

Check our website for further information: https://softstack.io

9. Glossary

Term	Definition
BIPS	Basis points; 1 bip = 0.01%
CEI	Checks-Effects-Interactions pattern for reentrancy prevention
CVSS	Common Vulnerability Scoring System
ERC-20	Ethereum-compatible fungible token standard supported by HTS tokens
ERC-4626	Tokenized vault standard whose share-accounting model the Vault is inspired by
FOK	Fill-Or-Kill; an order execution policy requiring full immediate fill or cancel
HAS	Hedera Account Service; precompile providing on-chain account-level signing and authorization primitives
HBAR	Native cryptocurrency of the Hedera network
HTS	Hedera Token Service; native Hedera token issuance and transfer system
Hedera	Public distributed ledger that uses the Hashgraph consensus algorithm
Hiero Hooks	Account-level on-chain programs that govern how assets in a Hedera account may be moved
IOC	Immediate-Or-Cancel; an order execution policy that fills as much as possible immediately and cancels the rest
MEV	Maximal Extractable Value; profit extracted by reordering or inserting transactions
OZ	OpenZeppelin, widely-used Solidity library for secure smart contracts
Order Hook	LambdaplexOrderHook — account-level order matching hook for spot trading
Settlement	LambdaplexSettlements — contract that finalizes signed orders and executes Hedera Token Service transfers
Supra Oracle	Third-party price oracle used by Lambdaplex for price proofs
TVL	Total Value Locked
Vault	PLEXPairVault — the balanced (BASE/QUOTE) liquidity vault
cryptoTransfer	HTS precompile call used by the Settlement contract to perform atomic multi-token transfers